

Parallel and Distributed Computing

Assignment 4

Submitted to: Dr Saif Nalband

Submitted by: Anushka Gupta

Roll No:102303358

Course: UCS645

Exercise 1: Ring Communication

Problem: Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 sends to Process 1, Process 1 sends to Process 2, and the last process sends back to Process 0.

Requirements:

Start with an initial value of 100 in Process 0

Each process adds its rank to the value before sending

Print the value at each process

Final value should return to Process 0

Hint: Use modulo operator to wrap around: $\text{next_rank} = (\text{rank} + 1) \% \text{size}$

Test with: `mpirun -np 4 ./ring_comm`

```
!mpicc -o ring_comm ring_comm.c
!mpirun --allow-run-as-root --oversubscribe -np 4 ./ring_comm
```

```
Process 0 starts with 100
Process 1 received 100
Process 1 after adding rank: 101
Process 2 received 101
Process 2 after adding rank: 103
Process 3 received 103
Process 3 after adding rank: 106
Process 0 received final value: 106
```

Exercise 2: Parallel Array Sum

Problem: Write an MPI program to compute the sum of an array in parallel. Divide the array among processes, compute local sums, and combine them.

```
!mpicc -o array_sum array_sum.c
!mpirun --allow-run-as-root --oversubscribe -np 4 ./array_sum
```

```
Global sum: 5050 (expected 5050)
```

Exercise 3: Finding Maximum and Minimum

Problem: Write an MPI program where each process generates random numbers, and the program finds the global maximum and minimum values across all processes.

Requirements:

Each process generates 10 random numbers (0-1000)

Each process finds its local maximum and minimum

Use MPI_Reduce with MPI_MAX and MPI_MIN

Process 0 prints the global maximum and minimum

Also print which process had these values (use MPI_MAXLOC/MPI_MINLOC)

Hint: For MPI_MAXLOC, use a struct with value and rank fields.

```
!mpicc -o max_min max_min.c
!mpirun --allow-run-as-root --oversubscribe -np 4 ./max_min
```

```
Global MAX: 992 from process 1
Global MIN: 41 from process 3
```

Exercise 4: Parallel Dot Product

Problem: Compute the dot product of two vectors in parallel.

Requirements:

Vector A = [1, 2, 3, 4, 5, 6, 7, 8]

Vector B = [8, 7, 6, 5, 4, 3, 2, 1]

Distribute vectors using MPI_Scatter

Each process computes partial dot product

Use MPI_Reduce to sum partial products

Expected result: 120

```
!mpicc -o dot_product dot_product.c
!mpirun --allow-run-as-root --oversubscribe -np 4 ./dot_product
```

```
Final Dot product: 120 (expected 120)
```

Conclusion:

All programs executed successfully on Google Colab using MPICH.